Bitácora de Análisis — Backend IcfesApp (quizAppIcfesBack)

Campo	Valor
Fecha del análisis	2026-10-26
Rama analizada	`App_test` (último commit `140b068 update app`)
Tipo de análisis	Estático (lectura de código, **sin ejecutar** la aplicación ni pruebas)
Alcance	Solución completa `quizAppIcfesBack.slnx` (4 proyectos) + configuración + migraciones. Además, revisión de la integración con el front Flutter (`QuizAppFront/quizz_application`, sección 12)
Documentos de referencia	`documento/Posible RFD.docx.pdf` (Documento de Requerimientos v1.0, actualizado en `documento/RFD-v2.0.md` / `.pdf`) y `documento/bitacora_Icfes.md` (estado y alcance del equipo)

---

1. Resumen ejecutivo

IcfesApp es el **backend (API REST) de una plataforma de preparación para las pruebas ICFES** (examen de Estado de Colombia). Está construido en **.NET 10 / ASP.NET Core** con **Entity Framework Core**, **ASP.NET Identity** y **JWT**, siguiendo una arquitectura limpia de 4 capas (Domain, Application, Infrastructure, API).

Lo que hoy permite hacer el sistema:

- Registrar usuarios (formulario o Google), confirmar correo, iniciar sesión con JWT + refresh token, recuperar/cambiar contraseña y usar 2FA (TOTP).
- Manejar dos modelos de acceso: **Particular** (paga un plan) e **Institucional** (entra con un código de invitación de su colegio).
- Administrar materias, preguntas de opción múltiple (con justificación), instituciones y códigos de invitación.
- Consultar catálogo de planes y suscripciones (solo lectura).
- Que un estudiante haga **sesiones de práctica** por materia: recibe preguntas aleatorias, responde, ve si acertó con la justificación y al final obtiene su puntaje.

Visión del equipo: un pre-ICFES **al estilo Duolingo**, con rutas de lecciones cortas, gamificación, **videos** y un **tutor IA** que explique las temáticas. La base técnica está avanzada, pero esos diferenciadores aún no se han empezado (ver sección 13).

Estado general: **base sólida y bien organizada**, con buen manejo de autenticación. Sin embargo, el módulo de suscripciones aún **no restringe el acceso**, no hay pagos, y se detectaron algunos **problemas de seguridad y de lógica** importantes (sección 9) que conviene corregir antes de llegar a producción.

---

2. Stack tecnológico

Componente	Tecnología / Paquete	Versión
Framework	.NET / ASP.NET Core	net10.0
ORM	Entity Framework Core (Sqlite + SqlServer)	10.0.10
Identidad	Microsoft.AspNetCore.Identity.EntityFrameworkCore	10.0.10
Autenticación	JwtBearer (HMAC-SHA256)	10.0.10
Login social	Google.Apis.Auth (validación de ID token)	1.75.0
Correo	MailKit (SMTP con STARTTLS)	4.17.0
Documentación API	Microsoft.AspNetCore.OpenApi + Swashbuckle (solo UI)	10.0.10 / 10.2.2
Base de datos activa	SQLite (`icfesapp.db`)	—

Características de C# usadas: *primary constructors*, `required`, colecciones `[]`, *raw string literals*, `Nullable` habilitado, `ImplicitUsings`.

---

3. Arquitectura

```
quizAppIcfesBack.slnx
└── src/
    ├── IcfesApp.Domain          → Entidades, enums, constantes (sin dependencias)
    ├── IcfesApp.Application     → DTOs, interfaces de servicios, Result/AuthResult
    │                              (depende de Domain + EF Core por DbSet<>)
    ├── IcfesApp.Infrastructure  → EF Core, Identity, JWT, SMTP, servicios, seeders,
    │                              migraciones (depende de Application)
    └── IcfesApp.API             → Program.cs, controladores, configuración
                                   (depende de Application + Infrastructure)
```

Flujo de una petición: Controlador (API) → interfaz de servicio (Application) → implementación (Infrastructure) → `ApplicationDbContext` (EF Core) → SQLite.

Patrones observados:

- **Clean Architecture / Onion**: el dominio no depende de nada; las interfaces viven en Application y se implementan en Infrastructure.
- **Inyección de dependencias** centralizada en `Infrastructure/DependencyInjection.cs` (`AddInfrastructure`).
- **Result pattern**: `Result`, `Result<T>`, `OperationResult` y `AuthResult` evitan excepciones para errores de negocio; los controladores traducen a `400/404/204`.
- **DTOs** separados de las entidades en todas las respuestas.
- **Configuraciones Fluent API** por entidad (`Persistence/Configurations/*`), cargadas con `ApplyConfigurationsFromAssembly`.
- `BaseEntity`: `Id` (Guid generado en C#), `CreatedAtUtc`, `UpdatedAtUtc`.

Observaciones de arquitectura:

- `Application` depende de `Microsoft.EntityFrameworkCore` porque `IApplicationDbContext` expone `DbSet<>`. Es un atajo pragmático habitual, pero acopla la capa de aplicación al ORM.
- `SubscriptionService` y `AuthService` usan directamente `ApplicationDbContext` (concreto) porque `Users` y `RefreshTokens` no están en la interfaz.
- Los servicios "de contenido" están en Infrastructure (no hay capa de casos de uso en Application); la lógica de negocio vive en Infrastructure.

---

4. Arranque de la aplicación (`Program.cs`)

1. `AddInfrastructure(configuration)`: BD, Identity, JWT, políticas y servicios.
2. `AddControllers()` y `AddOpenApi()` con dos *transformers*: define el esquema **Bearer** y lo aplica a **todas** las operaciones (incluidas las anónimas).
3. CORS **solo en Development**: permite cualquier puerto de `localhost` / `127.0.0.1`.
4. Pipeline: OpenAPI (`/openapi/v1.json`) + Swagger UI (`/swagger`) solo en Development → `UseHttpsRedirection` → `UseAuthentication` → `UseAuthorization` → `MapControllers`.
5. **Al iniciar, en cualquier entorno**: `Database.MigrateAsync()`, `RoleSeeder.SeedAsync()` y `DataSeeder.SeedAsync()`.

Configuración (`appsettings*.json`):

- `Database:Provider` = `Sqlite` (alternativa `SqlServer` con LocalDB).

- `Jwt`: issuer `IcfesApp`, audience `IcfesAppClients`, access token 15 min, refresh 14 días.
- `Email`: SMTP Gmail con credenciales *placeholder*; URLs del front en `localhost:4200`.
- `Authentication:Google:ClientId`: vacío.
- `launchSettings.json`: `https://localhost:7126` y `http://localhost:5080`, abre `/swagger`.

---

5. Modelo de dominio

5.1 Entidades

Entidad	Campos principales	Notas
`ApplicationUser` (Identity)	FirstName, LastName, `Type` (Particular/Institutional), `SchoolId?`	Hereda `IdentityUser<Guid>`. Al borrar el colegio → `SchoolId = null`.
`School` (expuesta como "institutions")	Name, Domain, City, IsActive, CreatedByUserId	`CreatedByUserId` nunca se llena.
`InvitationCode`	Code (único, máx. 50), InstitutionId, MaxUses (-1 = ilimitado), Uses, ExpiresAtUtc, IsActive	Propiedad calculada `IsUsable`. Cascade al borrar institución.
`Plan`	Code (único), Name, Target, DurationDays, Price (18,2), Currency (COP), IsActive, DisplayOrder	Catálogo.
`Subscription`	UserId? **o** InstitutionId?, PlanId, Status, StartedAtUtc, ExpiresAtUtc, CancelledAtUtc, AutoRenew, Notes	*Check constraint* `CK_Subscription_Owner`: exactamente un dueño.
`Subject`	Name, Description	Materia.
`Question`	Text, Justification, Difficulty, SubjectId, CreatedByUserId	Opciones en cascada.
`QuestionOption`	Text, IsCorrect, QuestionId	
`PracticeSession`	StudentUserId, SubjectId, Status, StartedAtUtc, FinishedAtUtc	Respuestas en cascada.
`StudentAnswer`	PracticeSessionId, QuestionId, SelectedOptionId?, IsCorrect, AnsweredAtUtc	FK a Question y Option con `Restrict`.
`RefreshToken`	Token (único), UserId, ExpiresAtUtc, RevokedAtUtc	`IsActive` calculado. Sin FK a usuarios.

5.2 Enums

- `UserType`: Particular = 0, Institutional = 1.
- `PlanTarget`: Particular = 0, Institutional = 1.
- `SubscriptionStatus`: Pending, Active, Expired, Cancelled, PastDue.
- `PracticeSessionStatus`: InProgress = 1, Completed = 2, Abandoned = 3.
- `QuestionDifficulty`: Easy = 1, Medium = 2, Hard = 3.

5.3 Roles y políticas

Política	Roles permitidos
`RequireAdmin`	Admin
`RequireTeacher`	Admin, Teacher
`RequireStudent`	Admin, Teacher, Student (definida pero **no se usa** en ningún controlador)

5.4 Migraciones (SQLite)

1 `20260929_InitialCreate`

2 `20261012_AddUserTypeToApplicationUser`

3 `20261012_SyncUserTypeEnumDefault`

4 `20261016_AddInvitationCodes`

5 `20261016_AddPlansAndSubscriptions`

Las migraciones están generadas con tipos de **SQLite** (`TEXT`). Si se cambia `Database:Provider` a `SqlServer`, estas migraciones no son compatibles: habría que generar un conjunto separado.

---

6. Catálogo de endpoints

6.1 Autenticación — `api/auth`

Método	Ruta	Acceso	Descripción
POST	`/register`	Público	Registro (siempre rol Student). Código de invitación opcional → Institucional. Envía correo de confirmación.
POST	`/register-staff`	Admin	Crea Teacher o Admin con correo ya confirmado y devuelve tokens.
POST	`/login`	Público	Email + contraseña. 401 credenciales, 403 correo sin confirmar, 423 bloqueado; si tiene 2FA devuelve `TwoFactorToken`.
GET	`/session`	Autenticado	Datos del usuario desde los *claims* del JWT.
POST	`/refresh`	Público	Rota el refresh token (revoca el anterior, emite par nuevo).
POST	`/revoke`	Autenticado	Revoca un refresh token (logout).
POST	`/forgot-password`	Público	Envía enlace de recuperación (respuesta genérica, no revela si existe el correo).
POST	`/reset-password`	Público	Restablece con token; revoca todos los refresh tokens.
POST	`/change-password`	Autenticado	Requiere contraseña actual; rechaza la misma; revoca refresh tokens.
POST	`/confirm-email`	Público	Confirma correo con token.
POST	`/resend-confirmation`	Público	Reenvía confirmación (respuesta genérica).
POST	`/google`	Público	Valida ID token de Google; crea el usuario si no existe (Student).
GET	`/2fa/setup`	Autenticado	Clave compartida + URI `otpauth://` para la app autenticadora.
POST	`/2fa/enable`	Autenticado	Verifica código TOTP, activa 2FA y devuelve 10 códigos de recuperación.
POST	`/2fa/disable`	Autenticado	Verifica TOTP, desactiva 2FA y resetea la clave.
POST	`/2fa/verify`	Público	Completa el login con `TwoFactorToken` + TOTP o código de recuperación.

6.2 Contenido y negocio

Controlador	Ruta base	Lectura	Escritura
`SubjectsController`	`api/subjects`	Autenticado	Admin (POST/PUT/DELETE)
`QuestionsController`	`api/questions` (`?subjectId=`)	Autenticado	Admin/Teacher
`InstitutionsController` (archivo `SchoolsController.cs`)	`api/institutions`	Autenticado	Admin
`InvitationCodesController`	`api/invitation-codes`	Admin	Admin (+ `POST /{id}/revoke`)
`PlansController`	`api/plans` (`?target=&includeInactive=`)	Anónimo	—
`SubscriptionsController`	`api/subscriptions`	`GET /current` autenticado; `GET /` y `GET /institution/{id}` Admin	—

Controlador	Ruta base	Lectura	Escritura
`PracticeSessionsController`	`api/practice-sessions`	Autenticado (solo sus sesiones)	`POST /` iniciar, `POST /{id}/answers`, `POST /{id}/finish`

---

7. Flujos de negocio principales

7.1 Registro con / sin código de invitación

1. Si viene `InvitationCode`, se normaliza (trim + mayúsculas), se busca y se valida `IsUsable` (activo, con cupo, no expirado).
2. Se crea el usuario: `Type = Institutional` y `SchoolId = InstitutionId` si hay código; si no, `Particular`.
3. En una **transacción**: `CreateAsync` + rol Student + `Uses++` del código.
4. Fuera de la transacción se envía el correo de confirmación (si falla, se registra en log; se puede reenviar).
5. Respuesta: "revisa tu correo". No se pueden iniciar sesión hasta confirmar.

7.2 Login y emisión de tokens

1. Busca por email → `CheckPasswordSignInAsync` con bloqueo (5 intentos → 15 min).
2. Verifica correo confirmado.
3. Si tiene 2FA → devuelve un **JWT de reto** (5 min, claim `purpose=2fa_challenge`).
4. Si no → `IssueTokensAsync`: access token JWT (sub, email, roles, `user_type`) + refresh token aleatorio de 64 bytes guardado en BD.

7.3 Refresh con rotación

Busca el refresh token, valida que esté activo, lo marca revocado y emite un par nuevo (sin datos de usuario para aligerar la respuesta).

7.4 Sesión de práctica

1. `POST /practice-sessions` con `SubjectId` y `QuestionCount` opcional.
2. Se cargan **todas** las preguntas de la materia a memoria, se barajan con `Guid.NewGuid()` y se toman N.
3. Se crea la sesión con un `StudentAnswer` vacío por pregunta (`IsCorrect = false`).
4. La respuesta oculta cuál opción es correcta (`PracticeQuestionOptionDto` sin `IsCorrect`).
5. `POST /{id}/answers`: valida pertenencia y estado, guarda la opción y devuelve si acertó, la opción correcta y la justificación.
6. `POST /{id}/finish`: marca `Completed` y devuelve total, aciertos, porcentaje y revisión por pregunta.

7.5 Suscripciones

- `GET /subscriptions/current`: si el usuario es Institucional devuelve la suscripción de su colegio; si es Particular, la propia más reciente.
- `HasActiveAccessAsync` existe y está documentado como "la función que se usará para gatear endpoints", **pero nadie lo llama todavía**.

---

8. Datos semilla (seeders)

`RoleSeeder`: crea roles Admin/Teacher/Student y, si no hay ningún Admin, crea `admin@icfesapp.com` / `Admin123!`.

`DataSeeder` (idempotente por tabla):

- 3 instituciones: Colegio San José (Bogotá), Liceo Moderno (Medellín), I.E. La Esperanza (Cali).
- 5 materias: Matemáticas, Lectura Crítica, Ciencias Naturales, Ciencias Sociales, Inglés.
- 7 usuarios con contraseña `Test12345!`: teacher1-2, student1-5 (student4 y student5 son Particulares).
- 3 códigos de invitación: `COLEGIO-SJ-2026`, `LICEOMODERNO-2026`, `PRUEBA-ILIMITADO`.
- 25 preguntas (5 por materia) con 4 opciones y justificación.
- 5 sesiones de práctica de ejemplo (completadas, en curso y abandonada).
- 4 planes: FREE (7 días, $0), STANDARD (30 días, $29.900), SEMESTRAL (180 días, $149.900), INSTITUTIONAL (365 días, $1.800.000).
- Suscripciones demo: San José y Liceo activas; La Esperanza sin suscripción; student4 STANDARD activo; student5 FREE vencido.

---

9. Hallazgos (problemas y riesgos)

Severidad: **Alta** = corregir antes de producción; **Media** = corregir pronto; **Baja** = mejora. Como el análisis es estático, los puntos marcados *"a verificar"* deben confirmarse ejecutando la app.

9.1 Seguridad

#	Sev.	Hallazgo	Dónde	Recomendación
S1	Alta	Los estudiantes pueden ver **las respuestas correctas**: `GET /api/questions` solo exige estar autenticado y `QuestionDto` incluye `IsCorrect` y `Justification`.	`QuestionsController`, `QuestionService.ToDto`	Restringir lectura del banco a Teacher/Admin, o usar un DTO sin `IsCorrect` para estudiantes.
S2	Alta	El **token de reto 2FA** se firma con la misma clave, issuer y audience que el access token. JwtBearer probablemente lo acepte como Bearer válido (el claim `sub` se mapea a `NameIdentifier`) y así se saltaría el 2FA en endpoints con `[Authorize]` sin rol. *A verificar.*	`JwtTokenService.CreateTwoFactorChallengeToken`	Usar otra audience/clave para el reto, o rechazar en `OnTokenValidated` los tokens con `purpose=2fa_challenge`.
S3	Alta	En producción también se ejecutan migraciones y **seeders con usuarios de prueba y contraseñas conocidas** (`Admin123!`, `Test12345!`).	`Program.cs`, `RoleSeeder`, `DataSeeder`	Ejecutar `DataSeeder` solo en Development; el admin inicial debe venir de variables de entorno/secretos.
S4	Alta	**Secreto JWT versionado** en `appsettings.Development.json` y la base `icfesapp.db` también está en git (contiene hashes y datos).	Repositorio	Mover a User Secrets (ya existe `UserSecretsId`) / variables de entorno; rotar el secreto; añadir `*.db` al `.gitignore` y sacarla del repo.
S5	Media	Sin **rate limiting** en login, registro, forgot-password y 2FA (exigido en RNF-SEG-03).	`Program.cs`	Usar `AddRateLimiter` de ASP.NET Core.
S6	Media	Refresh tokens guardados **en texto plano** y sin detección de reutilización (si un token revocado se usa, no se revoca la familia).	`RefreshToken`, `AuthService.RefreshAsync`	Guardar hash SHA-256; ante reuso, revocar todos los tokens del usuario.
S7	Media	`/refresh` no comprueba si el usuario está bloqueado, y `/google` no aplica la verificación de bloqueo.	`AuthService`	Revisar `IsLockedOutAsync` antes de emitir tokens.
S8	Baja	`/revoke` no comprueba que el refresh token pertenezca al usuario autenticado.	`AuthService.RevokeRefreshTokenAsync`	Filtrar por `UserId`.

#	Sev.	Hallazgo	Dónde	Recomendación
S9	Baja	Contraseñas con PBKDF2 (por defecto en Identity); el RFD pide Argon2/bcrypt (RNF-SEG-01).	Identity	Aceptable, o implementar un `IPasswordHasher` con Argon2.

9.2 Lógica de negocio

#	Sev.	Hallazgo	Dónde	Recomendación
L1	Alta	**Se puede responder la misma pregunta varias veces**: tras responder, la API revela la opción correcta, y el estudiante puede reenviar la correcta y sacar 100 %.	`PracticeSessionService.SubmitAnswerAsync`	Rechazar si `SelectedOptionId` ya tiene valor (o no revelar la correcta hasta finalizar).
L2	Alta	Las suscripciones **no restringen nada**: `HasActiveAccessAsync` no se usa; cualquier usuario autenticado practica sin plan.	`PracticeSessionsController`	Crear política/filtro "RequireActiveSubscription" que use `HasActiveAccessAsync`.
L3	Media	Editar una pregunta que ya tiene respuestas falla con **500**: se borran sus opciones, pero `StudentAnswer.SelectedOptionId` las referencia con `Restrict` (la excepción no se captura).	`QuestionService.UpdateAsync`	Actualizar opciones en sitio, versionar la pregunta (RF-QB-02) o capturar `DbUpdateException`.
L4	Media	Carrera en el contador de usos del código: dos registros simultáneos pueden superar `MaxUses` (la validación ocurre antes de la transacción).	`AuthService.RegisterAsync`	Token de concurrencia o `UPDATE ... WHERE Uses < MaxUses`.
L5	Media	Borrar una institución borra en **cascada** sus suscripciones (historial de facturación) y deja a sus usuarios Institucionales con `SchoolId = null` (sin acceso).	`SchoolConfiguration`/`SubscriptionConfiguration`	Usar desactivación (`IsActive=false`) en lugar de borrado físico.
L6	Media	El registro Particular **no crea ninguna suscripción** (ni trial FREE) ni existe endpoint para crear/cancelar suscripciones ni pagos.	Módulo suscripciones	Implementar trial FREE único (RF-SUB-07) y el módulo de pagos.
L7	Baja	`GET /auth/session` devuelve `FirstName` y `LastName` vacíos y no incluye `SchoolId`.	`AuthController.Session`	Consultar el usuario o añadir claims.
L8	Baja	`register-staff` crea Teachers sin institución y no distingue "Admin Institución" de "Admin Servidor" (el RFD los separa). Un Admin puede crear códigos para cualquier institución.	`AuthService.RegisterStaffAsync`	Añadir rol `InstitutionAdmin` y filtrar por su `SchoolId`.
L9	Baja	La sesión "en progreso" sembrada solo crea filas de respuestas contestadas, mientras que en tiempo real se crean todas por adelantado: datos demo inconsistentes.	`DataSeeder.CreateInProgressSession`	Crear todas las filas de respuesta.
L10	Baja	`Login` decide el código HTTP (403/423) **buscando texto en el mensaje de error** en español: es frágil ante cambios de redacción.	`AuthController.Login`	Agregar un código/enum de error en `AuthResult`.

9.3 Calidad, rendimiento y mantenimiento

#	Sev.	Hallazgo	Recomendación
C1	Media	**No hay proyecto de pruebas** (RNF-MAN-01 pide > 70 % de cobertura).	Crear `IcfesApp.Tests` (xUnit) con pruebas de servicios usando SQLite en memoria.

#	Sev.	Hallazgo	Recomendación
C2	Media	Sin **paginación** en listados (`questions`, `practice-sessions`, `subscriptions`, `invitation-codes`) — RNF-REN-04. `GetMine` carga todas las sesiones con preguntas y opciones.	Añadir `page/pageSize` y proyecciones.
C3	Media	`StartAsync` carga **todas las preguntas de la materia** a memoria para barajarlas.	Seleccionar solo Ids, barajar y luego cargar los N elegidos.
C4	Media	Sin manejador global de excepciones (`ProblemDetails`): errores no controlados devuelven 500 sin formato.	`AddProblemDetails()` + `UseExceptionHandler()`.
C5	Media	CORS solo en Development: el front Flutter Web en otro dominio no funcionará en producción.	Política CORS configurable por entorno.
C6	Baja	El selector `SqlServer` existe pero las migraciones son de SQLite.	Mantener migraciones por proveedor o elegir uno.
C7	Baja	Código duplicado: `CurrentUserId` y `ToActionResult` repetidos en varios controladores; mapeo de `PlanDto` duplicado; `IsUsable` recalculado en el DTO en vez de usar la propiedad de la entidad.	Controlador base o extensiones.
C8	Baja	El documento OpenAPI marca **todas** las operaciones como protegidas (incluidas las anónimas).	Aplicar el requisito solo si el endpoint tiene `[Authorize]`.
C9	Baja	Archivos sobrantes: `quizAppIcfesBack.http` apunta a `/weatherforecast` en el puerto 5002; `src/quizAppIcfesBack.lnk` (acceso directo de Windows) versionado; `README.md` casi vacío.	Limpiar y documentar.
C10	Baja	Nombres inconsistentes: entidad `School` / ruta `institutions` / archivo `SchoolsController.cs` con clase `InstitutionsController`.	Unificar nombre.
C11	Baja	`SchoolService.CreateAsync` no registra `CreatedByUserId` (campo de auditoría siempre nulo).	Pasar el usuario actual.

---

10. Cobertura frente al documento de requerimientos (RFD)

Requerimiento	Estado	Comentario
RF-AUTH-01/02 Registro y login	Hecho	Falta "tipo de usuario" explícito (se deduce del código).
RF-AUTH-03 Recuperación con token (24 h)	Hecho	Token de Identity (1 día por defecto).
RF-AUTH-04 Cambio de contraseña	Hecho	Revoca refresh tokens.
RF-AUTH-05 JWT + refresh con rotación	Hecho	Ver S6.
RF-AUTH-06 Cierre de sesión	Hecho	`/revoke`.
RF-AUTH-07/08 Registro particular / institucional	Parcial	Falta el flujo de planes tras el registro particular.
RF-AUTH-09 Menores con acudiente (Ley 1581)	Pendiente	No hay campos de consentimiento.
RF-AUTH-10 2FA	Hecho	Disponible para todos los roles. Ver S2.
RF-AUTH-11 Google	Hecho	Estaba marcado "Omitido" en el RFD, pero se implementó.
RF-SUB-01/02 Planes y estados	Hecho	Catálogo y enum completos.
RF-SUB-03…09 Gestión, cancelación, trial, renovación	Pendiente	Solo lectura.
RF-PAY-* Pagos (Stripe/Wompi, webhooks, IVA, DIAN)	Pendiente	No existe módulo de pagos.
RF-EST-01 Preguntas con explicación	Hecho	Justificación por pregunta.
RF-EST-02/08 Simulacro cronometrado / full	Pendiente	Solo práctica por materia sin tiempo.

Requerimiento	Estado	Comentario
RF-EST-03/04 Resultados e historial	Parcial	Puntaje y aciertos; falta tiempo y comparación.
RF-EST-05…07, 10, 11 Analítica, recomendaciones, contenido, offline, push	Pendiente	
RF-EST-09 Simulacro por materia	Parcial	Es la "sesión de práctica" actual.
Tutor IA	Pendiente (sin especificar)	Es parte central de la visión del equipo (13.1), pero no aparece en el RFD, en `bitacora_Icfes.md` ni en el spec del front. Lo más cercano: RF-EST-06 (recomendaciones) y RF-EST-07 (videos o avatar). Se especifica en la Fase 0 y se construye en la Fase 4.
RF-PROF-* Panel de profesor	Pendiente	El Teacher solo puede crear preguntas.
RF-ADM-01/02 Códigos de invitación	Hecho	Sin aislamiento por institución.
RF-ADM-03/04 Gestión de usuarios y roles	Parcial	Solo `register-staff`; no hay listar/editar usuarios.
RF-ADM-05 Banco de preguntas	Hecho	CRUD completo.
RF-ADM-08 Configuración de planes	Pendiente	Planes solo lectura.
RF-QB-01 Taxonomía (tema, subtema, año)	Parcial	Solo materia y dificultad; falta `Topic`.
RF-QB-02/04 Versionado e importación masiva	Pendiente	
RNF-SEG-02 JWT corto + rotación	Hecho	15 min / 14 días.
RNF-SEG-03 Rate limiting	Pendiente	
RNF-SEG-10 Bloqueo tras N intentos	Hecho	5 intentos / 15 min.
RNF-MAN-02 Swagger	Hecho	Solo en Development.
RNF-MAN-05 Migraciones versionadas	Hecho	5 migraciones.
RNF-MAN-01 Pruebas > 70 %	Pendiente	
RNF-REN-04 Paginación	Pendiente	

---

11. Coordinación con el documento de estado del equipo

Comparación entre `documento/bitacora_Icfes.md` (documento de estado y alcance preparado con el otro desarrollador) y el código real del backend.

Conclusión: el equipo está **coordinado en lo general** (visión B2C/B2B, lo implementado, credenciales y roadmap coinciden con el código), pero hay puntos donde el documento afirma más de lo que el backend hace hoy.

11.1 Puntos coincidentes

- Autenticación: JWT + refresh con rotación, recuperación de contraseña, confirmación de correo, cambio de contraseña con revocación de sesiones y roles Admin/Teacher/Student.
- Códigos de invitación con límite de usos, expiración y vínculo automático del estudiante con su institución (`SchoolId`).
- Catálogo de planes (FREE 7 días, STANDARD 30, SEMESTRAL 180, INSTITUTIONAL 365, en COP) y suscripción que puede pertenecer a un usuario o a una institución.
- Práctica por materia con preguntas aleatorias y feedback inmediato con justificación.
- Las credenciales de prueba existen en los seeders. Juan y Pepito, usados en el ejemplo del panel de profesor, son student1 y student2.
- Pendientes: panel de profesor, simulacro completo, Stripe, rol InstitutionAdmin, Ley 1581 y Wompi. Coinciden con la sección 10.

11.2 Diferencias detectadas

#	El documento de estado dice	Realidad en el backend	Acción sugerida
D1	"El plan FREE se oculta si el usuario ya consumió su prueba."	No existe esa regla en el backend. `GET /api/plans` es anónimo. `/subscriptions/current` solo devuelve la suscripción **más reciente**: si el usuario usó FREE y luego compró STANDARD, el front no puede saber que ya gastó el FREE.	Crear endpoint `GET /api/plans/available` (autenticado) o un campo `HasUsedFreeTrial` en el backend.
D2	"Estudiante (Expirado): su plan FREE ya expiró."	La API **no bloquea** por suscripción: student5 puede practicar normalmente (ver L2).	Implementar el control con `HasActiveAccessAsync`; mientras tanto, no mostrar este caso en la demo.
D3	"Dashboard con resumen de progreso" e "Historial de prácticas".	No hay endpoint de estadísticas. `GET /practice-sessions` no trae puntaje ni aciertos; el resultado solo se obtiene con `POST /{id}/finish` (un POST usado para leer).	Agregar `CorrectAnswers` y `ScorePercentage` a `PracticeSessionDto` o crear `GET /api/practice-sessions/stats`. Confirmar cómo lo arma el front.
D4	"2FA preparada."	El 2FA está **completo** (setup, enable, disable, verify y códigos de recuperación).	Actualizar el documento.
D5	No menciona login con Google.	`/api/auth/google` existe, pero `ClientId` está vacío y no funciona. El RFD lo marcaba "Omitido".	Decidir si se usa o se retira.
D6	Estados: Active, Expired, Pending, Cancelled.	El backend también tiene `PastDue`.	Actualizar el documento.
D7	"Fácil transición futura a PostgreSQL/MySQL."	Solo hay soporte para SQLite y SQL Server, y las migraciones son específicas de SQLite (C6).	Matizar: requiere otro paquete y migraciones nuevas.
D8	"Contratos documentados con Swagger/OpenAPI."	Swagger solo está disponible en Development.	Aclararlo en el documento.
D9	Ley 1581 en prioridad media.	El RFD la ubica como Alta y en la Fase 1 del MVP.	Actualizar el RFD con la nueva priorización (B2B primero).
D10	Sección "Cómo ejecutar el proyecto localmente".	El archivo está **cortado**: termina en `dotnet run` sin cerrar el bloque y sin instrucciones del front. No indica los puertos.	Completar: API en `http://localhost:5080` y `https://localhost:7126`, Swagger en `/swagger`.

11.3 Temas ausentes en el roadmap del equipo

- Los estudiantes pueden ver las respuestas correctas vía `GET /api/questions` (S1).
- Se puede responder la misma pregunta varias veces y obtener 100 % (L1).
- Los seeders con contraseñas conocidas corren en producción, y la clave JWT y `icfesapp.db` están en git (S3, S4).
- El panel de profesor requiere **endpoints nuevos**: hoy un profesor no puede consultar resultados de estudiantes; cada estudiante solo ve sus propias sesiones.

11.4 Sobre "listo para Demo Institucional"

Sí, con una advertencia. El guion propuesto funciona con el backend actual (student1 practica y el admin genera un código en vivo). Debe evitarse mostrar en vivo el caso del estudiante con plan vencido (student5), porque la API no lo bloquea.

---

12. Coordinación Front (Flutter) ↔ Back

Revisión del repositorio del front `D:\proyectos\QuizAppFront\quizz_application` (rama local, 5 commits) para comprobar que front y back están coordinados.

Alcance de esta revisión: se leyó toda la capa que habla con la API (configuración, cliente HTTP, almacenamiento de tokens, autenticación, repositorios y modelos), el router, las páginas de planes, suscripción, práctica, dashboard del estudiante y panel administrativo/docente, y el documento de especificación del front `SpecDriven_QuizApp.md`. No es una auditoría visual ni de calidad de todo el código Flutter. Tampoco se ejecutó la app.

12.1 Resumen del front

Aspecto	Detalle
Stack	Flutter (SDK ^3.12), `http`, `flutter_secure_storage`, `go_router`
Tamaño	~10.000 líneas en `lib/`, organizado por funcionalidades (`features/`)
URL de la API	`API_BASE_URL` por `--dart-define`; por defecto `https://localhost:7126` (coincide con el perfil `https` del backend)
Sesión	Tokens en almacenamiento seguro; refresh automático ante 401 con una sola petición a la vez; al recargar llama a `/api/auth/refresh` y luego a `/api/auth/session`
Pruebas	4 archivos, incluida una prueba de contrato de la API (`frontend_api_contract_test.dart`)
Pantallas	Login, registro (con código de invitación), confirmación de correo, recuperación y cambio de contraseña, dashboard del estudiante, práctica, planes, "Mi suscripción" y panel administrativo/docente

12.2 Lo que está bien coordinado

- **Todos los endpoints que usa el front existen** en el backend, con las mismas rutas, verbos y nombres de campos (auth, materias, instituciones, preguntas, códigos de invitación, planes, suscripción actual y sesiones de práctica).
- Los modelos del front leen exactamente los campos de los DTOs del backend (por ejemplo `expiresAtUtc`, `refreshTokenExpiresAtUtc`, `isUsable`, `daysRemaining`, `questionId`, `correctOptionId`).
- Los errores se leen del formato `{ errors: [...] }` que usa el backend, y también del formato de validación de ASP.NET.
- Crear un profesor desde el panel **no reemplaza la sesión del admin** (el front ignora los tokens que devuelve `register-staff`); hay una prueba para eso.
- La práctica nunca muestra la opción correcta antes de responder y deshabilita las opciones después de responder.
- El front es honesto con lo que falta en el backend: muestra "Sin endpoint global" en "Estudiantes activos", "El backend actual no expone un listado de usuarios" en Profesores, y "Próximamente" al elegir un plan de pago.
- Las rutas `/confirm-email` y `/reset-password` del front coinciden con los enlaces que arma el backend en los correos.

12.3 Diferencias detectadas

#	Sev.	Diferencia	Dónde	Acción sugerida
F1	Alta	**Al recargar la página se pierden el nombre y el colegio del usuario.** El front restaura la sesión con `GET /api/auth/session`, y el backend devuelve `firstName` y `lastName` vacíos y no envía `schoolId`.	Back: `AuthController.Session` / Front: `AuthRepository.restoreSession`	Completar `/auth/session` en el backend (L7).

#	Sev.	Diferencia	Dónde	Acción sugerida
F2	Alta	**Los enlaces de los correos apuntan a** `http://localhost:4200`, pero nada en el front fija ese puerto. Si Flutter Web corre en otro puerto, confirmar el correo o restablecer la contraseña no funciona, y sin confirmar el correo no se puede iniciar sesión.	Back: `EmailSettings` / Front: comando de ejecución	Ejecutar el front con `--web-port 4200` o configurar las URL del correo por entorno.
F3	Alta	**Nadie bloquea por suscripción.** Ni el backend ni el router o el dashboard del front revisan si el plan está vigente: student5 (plan vencido) practica normalmente. Confirma D2.	Ambos	Bloqueo en el backend (L2) y pantalla de "plan vencido" en el front.
F4	Media	**La regla "ocultar FREE si ya lo usó" está en el front y falla.** `PlansPage` solo revisa la suscripción más reciente: si el usuario usó FREE y luego compró STANDARD, el FREE vuelve a aparecer. Confirma D1.	Front: `plans_page.dart`	Mover la regla al backend (campo `HasUsedFreeTrial` o endpoint de planes disponibles).
F5	Media	**El dashboard y el historial no muestran puntajes**, solo conteos ("Prácticas completas", "Respuestas enviadas", "x/y respondidas"), porque el backend no los envía. Confirma D3.	Front: `student_dashboard_page.dart`	Agregar puntaje y aciertos al listado en el backend; luego mostrarlos en el front.
F6	Media	**El login con 2FA rompe el front.** El backend responde `{ requiresTwoFactor, twoFactorToken }` sin `user`, y el front lo interpreta como "La respuesta de autenticación es inválida". Tampoco hay pantalla para activar el 2FA.	Front: `AuthResponse.fromJson`	Manejar `requiresTwoFactor` y agregar la pantalla de verificación (y de configuración) del 2FA.
F7	Media	**Editar una institución borra su dominio y la reactiva.** El front envía solo `name` y `city`; el backend pone `Domain = null` e `IsActive = true` por defecto.	Front: `updateSchool` / Back: `SchoolService.UpdateAsync`	Enviar `domain` e `isActive` desde el front, o que el backend no sobrescriba campos ausentes.
F8	Media	**Las prácticas no se pueden retomar ni abandonar.** El front siempre inicia una sesión nueva y no usa `GET /practice-sessions/{id}`; el backend no tiene endpoint para abandonar. Las sesiones a medias se acumulan como "En progreso".	Ambos	Retomar desde el historial y agregar un endpoint para abandonar.
F9	Baja	Las sesiones con estado `Abandoned` aparecen como "En progreso" en el front (solo distingue `Completed`).	Front: `PracticeSessionModel.isCompleted`	Manejar los tres estados.
F10	Baja	El profesor entra al "Panel docente", que solo gestiona el banco de preguntas: no hay reportes porque el backend no tiene endpoints (tarea 36, Fase 5 del plan). Al crear profesores no se elige colegio, así que quedan sin `SchoolId`.	Ambos	Ver tarea del panel de profesor en el backlog.
F11	Baja	El reto diario siempre usa la **primera materia** con 10 preguntas; no hay lógica de reto en el backend.	Front: `_startDailyChallenge`	Definir la regla del reto (o quitarlo hasta tenerla).

#	Sev.	Diferencia	Dónde	Acción sugerida
F12	Baja	`config/env.example.json` apunta a `https://api.example.com` y el repo del front no tiene README.	Front	Documentar cómo ejecutar el front contra el backend local.

12.4 Diferencias entre la especificación del front y el backend

El documento `SpecDriven_QuizApp.md` define roles y funciones que no coinciden con lo implementado en el backend:

Tema	Spec del front	Backend actual
Admin y contenido	"No administra materias ni preguntas."	El Admin puede crear materias (solo él) y preguntas. El panel del front también le permite crear materias.
Materias	Las mantiene el equipo de desarrollo directamente en la base de datos.	Hay CRUD de materias expuesto para el Admin.
Admin y usuarios	Crear, consultar, editar y **desactivar** profesores; consultar estudiantes, notas y promedios.	Solo existe `register-staff` (crear). No hay listado, edición, desactivación ni consulta de notas.
Profesor	Consultar estudiantes, notas y promedios.	No hay endpoints.
Estudiante	"Quizzes evaluativos" distintos de la práctica; consultar notas y promedios.	Solo existe la práctica; no hay quiz evaluativo ni promedios.
Gamificación	Racha, experiencia, nivel y reto diario desde el backend.	No existe nada de esto en el backend.
Respuesta correcta	"No incluir la respuesta correcta en datos expuestos al cliente cuando comprometa la evaluación."	`GET /api/questions` la expone (S1) y la práctica la revela al responder (L1).

Además, las casillas de "Próximos pasos" del spec están desactualizadas: por ejemplo, "Consumir materias reales desde el backend" sigue sin marcar, pero ya está hecho.

Recomendación: que el equipo acuerde **una sola fuente de verdad** para roles y permisos (RFD, `bitacora_Icfes.md` y `SpecDriven_QuizApp.md` dicen cosas distintas) y ajuste el backend y el front a esa definición.

---

13. Plan de trabajo por fases (orden de prioridades)

Esta sección reúne **todo lo pendiente**: hallazgos del código del backend (sección 9), requerimientos del RFD sin cubrir (sección 10), diferencias con el documento del equipo (sección 11) y con el front (sección 12), y lo que falta para llegar a la **visión del producto**. Es un **plan de ejecución**: las fases se hacen en orden, y dentro de cada fase las tareas también están en el orden recomendado. La fase de una tarea **es su prioridad**.

13.1 Visión del producto

El objetivo del equipo es un **pre-ICFES al estilo Duolingo**: intuitivo, con aprendizaje en pasos cortos y motivación diaria, que **no se base en lecturas aburridas** sino en **videos** y en un **tutor IA** que explique las temáticas.

Qué significa esto en funcionalidades concretas:

Pilar	Qué incluye
Ruta de aprendizaje	Materias divididas en temas, unidades y lecciones cortas (5–10 preguntas), que se desbloquean en orden.
Motivación (gamificación)	Experiencia (XP), racha diaria, niveles, meta diaria y reto diario; más adelante ligas o ranking.
Contenido en video	Videos cortos por tema, vistos antes o durante la lección.
Tutor IA	Explica la temática de cada lección y responde "¿por qué?" cuando el estudiante se equivoca, siempre dentro del temario ICFES.
Evaluación	Práctica con feedback inmediato (ya existe) y simulacro completo cronometrado.
Institucional	Reportes del profesor y del colegio sobre el progreso de sus estudiantes.

13.2 Brecha entre lo construido y la visión

Pilar	Estado	Comentario
Motor de práctica con feedback inmediato y justificación	Construido	Es la base de las lecciones. Hay que corregir L1 (responder dos veces).
Materias y banco de preguntas	Construido	Falta la división en temas (`Topic`) y la carga masiva de contenido.
Autenticación, colegios, códigos y planes	Construido	Faltan las correcciones de las fases 1 y 2.
Ruta de aprendizaje (temas, unidades, lecciones)	No existe	No hay entidades ni pantallas.
Gamificación (XP, racha, niveles)	No existe	El front tiene la tarjeta de reto diario, pero sin lógica.
Videos	No existe	
Tutor IA	No existe	No está especificado en ningún documento (ver 10 y Fase 0).
Reportes del profesor	No existe	Depende de los puntajes y de los temas.

Conclusión: la base técnica (autenticación, práctica, administración) está bastante avanzada, pero **lo que diferencia al producto todavía no empieza**. Por eso el plan pone la experiencia de aprendizaje y el tutor IA **antes** de los módulos de venta.

13.3 Criterios usados para ordenar

1. **Decidir antes de construir:** las definiciones de producto (ruta de aprendizaje, gamificación, alcance del tutor IA, roles) van primero, porque cambian el diseño de todo lo demás.
2. **Estabilizar lo que existe antes de agregar funcionalidades:** primero seguridad y errores que rompen los flujos actuales.
3. **Base técnica antes de crecer:** pruebas, manejo de errores y paginación, para que lo nuevo nazca probado.
4. **El diferenciador antes que la venta:** la experiencia estilo Duolingo y el tutor IA son el producto; los reportes, pagos y licencias se apoyan en ellos (por ejemplo, el profesor ve el progreso por tema y por lección).
5. **Respetar dependencias:** cada tarea indica de cuáles depende.
6. **Legal antes de usuarios reales:** la Ley 1581 se cumple antes de que entren estudiantes reales (menores), y aplica también a los datos que se envíen al tutor IA.

Si hay una demo cercana: preparar un "corte de demo" con la Fase 1 completa, una materia con su ruta de aprendizaje y gamificación (Fase 3), un prototipo del tutor IA (Fase 4) y una versión simple del reporte del profesor (tarea 36).

13.4 Resumen de fases

Fase	Objetivo	Hito de cierre (cuándo se da por terminada)
0	Decisiones de producto	Hay un documento único con roles, reglas de negocio, diseño de la ruta de aprendizaje, gamificación y alcance del tutor IA, aprobado por el equipo.
1	Estabilizar y asegurar lo existente	No quedan hallazgos críticos de seguridad y los flujos actuales funcionan sin errores.
2	Base técnica	Hay pruebas automáticas, errores uniformes, límite de intentos y listados paginados.
3	Experiencia de aprendizaje estilo Duolingo	Un estudiante sigue una ruta de lecciones por tema, ve videos, gana XP, mantiene racha y ve su progreso.
4	Tutor IA	El tutor explica temas y errores dentro de las lecciones, con límites de uso y protección de datos de menores.
5	Producto institucional (B2B)	La demo y la venta a colegios están completas: reportes del profesor, simulacro y control por suscripción.
6	Piloto con colegios reales	Estudiantes reales usan la app cumpliendo la Ley 1581, con administración por colegio y entorno de producción.

Fase	Objetivo	Hito de cierre (cuándo se da por terminada)
7	Venta a particulares (B2C)	Un particular puede comprar, renovar y cancelar un plan con pago real.
8	Crecimiento y escala	Ligas, notificaciones, app móvil offline, avatar y limpieza técnica.

13.5 Fase 0 — Decisiones de producto (equipo, sin código)

#	Tarea	Ref.	Repo
1	Definir una sola tabla de roles y permisos (Admin servidor, Admin institución, Profesor, Estudiante): hoy el RFD, `bitacora_Icfes.md` y `SpecDriven_QuizApp.md` se contradicen.	12.4, L8	Equipo
2	Diseñar la **ruta de aprendizaje**: cómo se dividen las materias en temas, unidades y lecciones, cuántas preguntas tiene una lección y cómo se desbloquea la siguiente.	13.1	Equipo
3	Definir la **gamificación**: cómo se gana XP, qué rompe la racha, niveles, meta diaria y regla del reto diario.	F11, 12.4	Equipo
4	Especificar el **tutor IA**: qué explica (temas, errores, chat libre o guiado), proveedor de IA, límite de uso por plan, costo esperado, qué datos del estudiante puede recibir y cómo se evita que hable de temas fuera del ICFES.	RF-EST-06	Equipo
5	Definir los **videos y el contenido**: quién los produce, dónde se alojan (YouTube no listado, Vimeo o almacenamiento propio) y cómo se garantizan los derechos de uso de videos y preguntas.	RF-EST-07, RNF-LEG-07	Equipo
6	Resolver las decisiones pendientes del RFD: bloqueo o downgrade al expirar el plan, si un profesor pertenece a uno o varios colegios, trial FREE por email o por documento, banco de preguntas global o por institución, y si se usa el login con Google.	RFD §7, D5	Equipo

13.6 Fase 1 — Estabilizar y asegurar lo existente

#	Tarea	Ref.	Repo
7	Ocultar `IsCorrect` (y la justificación) en `GET /api/questions` para estudiantes, o restringir ese endpoint a Teacher/Admin.	S1	Back
8	Impedir responder dos veces la misma pregunta en una sesión de práctica (el front ya lo impide en pantalla, pero la API no).	L1	Back
9	Separar el token de reto 2FA del access token (otra audience o clave) y rechazarlo como Bearer. Verificarlo primero ejecutando la app.	S2	Back
10	Ejecutar `DataSeeder` solo en Development y tomar el admin inicial de variables de entorno o secretos.	S3	Back
11	Sacar la clave JWT de `appsettings.Development.json` (usar User Secrets), rotarla, añadir `*.db` al `.gitignore` y quitar `icfesapp.db` del repositorio.	S4	Back
12	Completar `GET /api/auth/session` con nombre, apellido y `SchoolId` (el front lo usa al recargar).	L7, F1	Back
13	Hacer que funcionen los enlaces de confirmación y recuperación: ejecutar el front en el puerto 4200 o configurar las URL del correo por entorno.	F2	Front + Back
14	Corregir el error 500 al editar una pregunta que ya tiene respuestas.	L3	Back
15	Editar una institución sin perder su dominio ni reactivarla.	F7	Back + Front
16	Mostrar correctamente las prácticas abandonadas en el front.	F9	Front

13.7 Fase 2 — Base técnica

#	Tarea	Ref.	Repo
17	Proyecto de pruebas del backend (xUnit) empezando por `AuthService` y `PracticeSessionService`; ampliar las pruebas de contrato del front.	C1, RNF-MAN-01	Back + Front
18	Manejo global de errores con `ProblemDetails`.	C4	Back

#	Tarea	Ref.	Repo
19	Rate limiting en login, registro, recuperación de contraseña y 2FA (luego se reutiliza para limitar el tutor IA).	S5, RNF-SEG-03	Back
20	CORS configurable por entorno para que Flutter Web funcione fuera de `localhost`.	C5	Back
21	Paginación en los listados y no cargar todas las preguntas a memoria al iniciar una práctica.	C2, C3, RNF-REN-04	Back + Front
22	Endurecer la sesión: refresh tokens guardados como hash, detección de reutilización, validar bloqueo en `/refresh` y `/google`, y que `/revoke` verifique al dueño del token.	S6, S7, S8	Back
23	Reemplazar la detección de errores por texto en el login por códigos de error.	L10	Back + Front

13.8 Fase 3 — Experiencia de aprendizaje estilo Duolingo

#	Tarea	Ref.	Depende de	Repo
24	Taxonomía del contenido: entidad `Topic` (tema/subtema) y año, y asociar cada pregunta a un tema.	RF-QB-01	2	Back + Front
25	Carga masiva de preguntas (CSV/Excel) para poblar los temas: una ruta de aprendizaje necesita mucho contenido.	RF-QB-04	24	Back + Front
26	**Ruta de aprendizaje**: unidades y lecciones por tema, progreso por lección y desbloqueo de la siguiente; pantalla tipo "camino" en el front. Reutiliza el motor de práctica actual.	13.1	2, 24	Back + Front
27	Puntaje y aciertos en el historial y en cada lección (base del progreso, de la gamificación y de los reportes).	D3, F5, RF-EST-03/04	—	Back + Front
28	**Gamificación**: XP por lección, racha diaria, niveles, meta diaria y reto diario real (reemplaza la tarjeta actual que siempre usa la primera materia).	F11, 13.1	3, 26, 27	Back + Front
29	**Videos por tema**: entidad de contenido de estudio con enlace al video, reproductor en la lección y registro de "visto".	RF-EST-07	5, 24	Back + Front
30	Retomar y abandonar lecciones y prácticas: usar `GET /practice-sessions/{id}` en el front y agregar un endpoint para abandonar.	F8	—	Back + Front

13.9 Fase 4 — Tutor IA

#	Tarea	Ref.	Depende de	Repo
31	Servicio de tutor IA **en el backend** (la clave del proveedor nunca va en la app Flutter), con instrucciones que lo limiten al temario ICFES y a un lenguaje adecuado para menores.	13.1	4, 24	Back
32	Explicación de errores: botón "¿Por qué?" en la retroalimentación de cada pregunta, que amplía la justificación con la ayuda del tutor. Guardar en caché las explicaciones por pregunta para reducir costos.	13.1	31	Back + Front
33	Explicación de temáticas: el tutor presenta el tema al iniciar una lección y responde dudas sobre él.	13.1	26, 31	Back + Front
34	Límites y control: cantidad de mensajes por día según el plan, registro de uso y costos, y no enviar datos personales del estudiante al proveedor (Ley 1581).	RNF-LEG-01	19, 31	Back
35	Recomendaciones de temas a reforzar según los errores del estudiante, sugeridas por el tutor.	RF-EST-06	27, 31	Back + Front

13.10 Fase 5 — Producto institucional (B2B)

#	Tarea	Ref.	Depende de	Repo
36	**Reportes del profesor**: listar los estudiantes de su colegio, ver resultados y progreso por materia, tema y fecha, y ordenar por porcentaje; asignar colegio al crear profesores; pantalla de reportes en el panel docente.	RF-PROF-01/02/03 /06, F10	1, 21, 27	Back + Front
37	Bloquear el acceso sin suscripción activa (propia o del colegio) y mostrar en el front una pantalla de "plan vencido".	L2, D2, F3	6	Back + Front
38	**Simulacro completo cronometrado** con las 5 materias (el "quiz evaluativo" del spec del front), con resultados por materia.	RF-EST-02/08	27	Back + Front
39	Evolución en el tiempo de un estudiante, visible para él y para su profesor, y comparación entre intentos.	RF-PROF-04, RF-EST-04/05	36	Back + Front
40	**Grupos o cursos** (11-A, 11-B…) para filtrar los reportes del profesor.	RF-PROF-02	36	Back + Front
41	Exportar los reportes del profesor a PDF o Excel.	RF-PROF-05	36	Back + Front
42	Rol **InstitutionAdmin** que gestione solo los códigos y usuarios de su colegio (aislamiento por institución).	L8, RF-ADM-01…03	1	Back + Front
43	Gestión de usuarios para el Admin: listar, editar y desactivar profesores y estudiantes (el front ya tiene la sección esperando el endpoint).	RF-ADM-04	1	Back + Front

13.11 Fase 6 — Piloto con colegios reales

#	Tarea	Ref.	Depende de	Repo
44	Cumplimiento **Ley 1581**: consentimiento del acudiente para menores, aceptación de términos y política de privacidad (incluye el uso del tutor IA).	RF-AUTH-09, RNF-LEG-01…04	—	Back + Front
45	Integridad de datos: no superar el cupo de los códigos con registros simultáneos y desactivar instituciones en lugar de borrarlas.	L4, L5	—	Back
46	Manejar el 2FA en el front (verificación en el login y activación), al menos para los administradores.	F6, RF-AUTH-10	9	Front
47	Aplicar la decisión sobre el login con Google (configurarlo en back y front, o retirarlo).	D5	6	Back + Front
48	Entornos separados (dev/staging/prod), CI/CD, base de datos de producción con sus propias migraciones y alojamiento de los videos.	C6, D7, RNF-MAN-03/04	17	Back + Front
49	Documentación: README del front, `env.example.json` con la URL real y actualización del spec del front con la visión estilo Duolingo.	F12	—	Front

13.12 Fase 7 — Venta a particulares (B2C)

#	Tarea	Ref.	Depende de	Repo
50	Ciclo de suscripción particular: trial FREE único controlado por el backend (reemplaza la regla actual del front), crear y cancelar suscripción, y aplicar la regla de expiración.	L6, D1, F4, RF-SUB-03…07	6, 37	Back + Front
51	**Pagos con Stripe**: checkout, webhooks firmados, idempotencia, log de auditoría, IVA del 19 % y páginas de resumen y resultado del pago.	RF-PAY-01/03…09, RNF-SEG-06	50	Back + Front
52	Integración con **Wompi** (PSE, Nequi, tarjetas).	RF-PAY-02	51	Back + Front

#	Tarea	Ref.	Depende de	Repo
53	Configuración de planes y precios desde el Admin (incluido el límite de uso del tutor IA por plan).	RF-ADM-08	34	Back + Front
54	Renovación automática y avisos antes del vencimiento.	RF-SUB-08/09	51	Back
55	Facturación electrónica DIAN.	RF-PAY-10, RNF-LEG-05	51	Back

13.13 Fase 8 — Crecimiento y escala

#	Tarea	Ref.	Depende de	Repo
56	Ligas o ranking anónimo entre estudiantes, con consentimiento.	RF-PROF-07	28, 44	Back + Front
57	Notificaciones push y por correo para no perder la racha y recordar la meta diaria.	RF-EST-11	28	Back + Front
58	App móvil con modo offline y sincronización posterior.	RF-EST-10	26	Front + Back
59	Avatar animado para el tutor IA.	RF-EST-07	33	Front
60	Versionado de preguntas.	RF-QB-02	24	Back
61	Limpieza técnica: código duplicado en controladores, OpenAPI marcando todo como protegido, `.http` obsoleto, `.lnk` versionado, README del backend, nombres School/Institution, `CreatedByUserId` en instituciones y datos de prueba de la sesión "en progreso".	C7…C11, L9	—	Back
62	Evaluar Argon2/bcrypt para las contraseñas.	S9, RNF-SEG-01	—	Back

---

14. Historial del proyecto (git)

Fecha	Commit	Descripción
2026-08-04	`d53c1bc` / `9f12333`	Commit inicial y *scaffold* del backend
2026-08-04	`3010660`	Configuración de CORS
2026-08-04	`be19a26`	JWT
2026-08-04	`ef16f60`	Roles y recuperación de contraseña
2026-08-07	`0c069b4` / `dfda18d`	CRUDs
2026-08-07	`52211de`	2FA
2026-08-07	`edcdbef`	Registro con Google y confirmación de correo
2026-10-07	`4b668b2`	SQLite y configuración de Swagger
2026-10-26	`23e6929` / `140b068`	Actualizaciones (tipos de usuario, códigos de invitación, planes y suscripciones)

---

15. Registro de la bitácora

Fecha	Actividad	Responsable
2026-10-26	Análisis estático completo del backend y comparación con el RFD v1.0. Creación de esta bitácora (.md y .pdf).	Claude Code (asistente) para AlejandroGiron97
2026-10-26	Comparación con el documento de estado del equipo (`bitacora_Icfes.md`): 10 diferencias y 4 temas ausentes del roadmap (sección 11).	Claude Code (asistente) para AlejandroGiron97
2026-10-26	Backlog consolidado de 40 tareas ordenadas por prioridad P0–P3 (luego reemplazado por la sección 13). Actualización de `bitacora_Icfes.md` con las correcciones de la sección 11.	Claude Code (asistente) para AlejandroGiron97

Fecha	Actividad	Responsable
2026-10-26	Revisión del front Flutter (`quizz_application`) frente al backend: 12 diferencias (F1–F12) y contradicciones con su spec (sección 12). Backlog ampliado a 51 tareas con columna de repo, incluidos los reportes de profesor faltantes.	Claude Code (asistente) para AlejandroGiron97
2026-10-26	Backlog reorganizado como **plan de trabajo por fases** (0 a 7) con criterios de orden, hitos de cierre y dependencias (sección 13). Se verificó que el **tutor IA** no está especificado en ningún documento y se agregó como decisión de la Fase 0.	Claude Code (asistente) para AlejandroGiron97
2026-10-26	Se incorporó la visión del producto (estilo Duolingo, videos y tutor IA). Plan reorganizado en 9 fases (0 a 8) con 62 tareas: la experiencia de aprendizaje y el tutor IA pasan antes de los módulos de venta (sección 13).	Claude Code (asistente) para AlejandroGiron97
2026-10-26	Análisis del RFD v1.0: se decidió **no eliminarlo** (es la única fuente de requerimientos y sus IDs se usan en esta bitácora) y se creó la **versión 2.0** (`RFD-v2.0.md` / `.pdf`) con estado y fase por requerimiento, la visión estilo Duolingo y los módulos nuevos de ruta de aprendizaje, gamificación, videos y tutor IA.	Claude Code (asistente) para AlejandroGiron97
	(siguiente entrada)	